

Chapter – 6

Public-Key Cryptography

Private-Key Cryptography

- traditional **private/secret/single key** cryptography uses **one** key
- shared by both sender and receiver
- if this key is disclosed communications are compromised
- also is **symmetric**, parties are equal
- hence does not protect sender from receiver forging a message & claiming is sent by sender

Public-Key Cryptography

- probably most significant advance in the 3000 year history of cryptography
- uses **two** keys – a public & a private key
- **asymmetric** since parties are **not** equal
- uses clever application of number theoretic concepts to function
- complements **rather than** replaces private key crypto

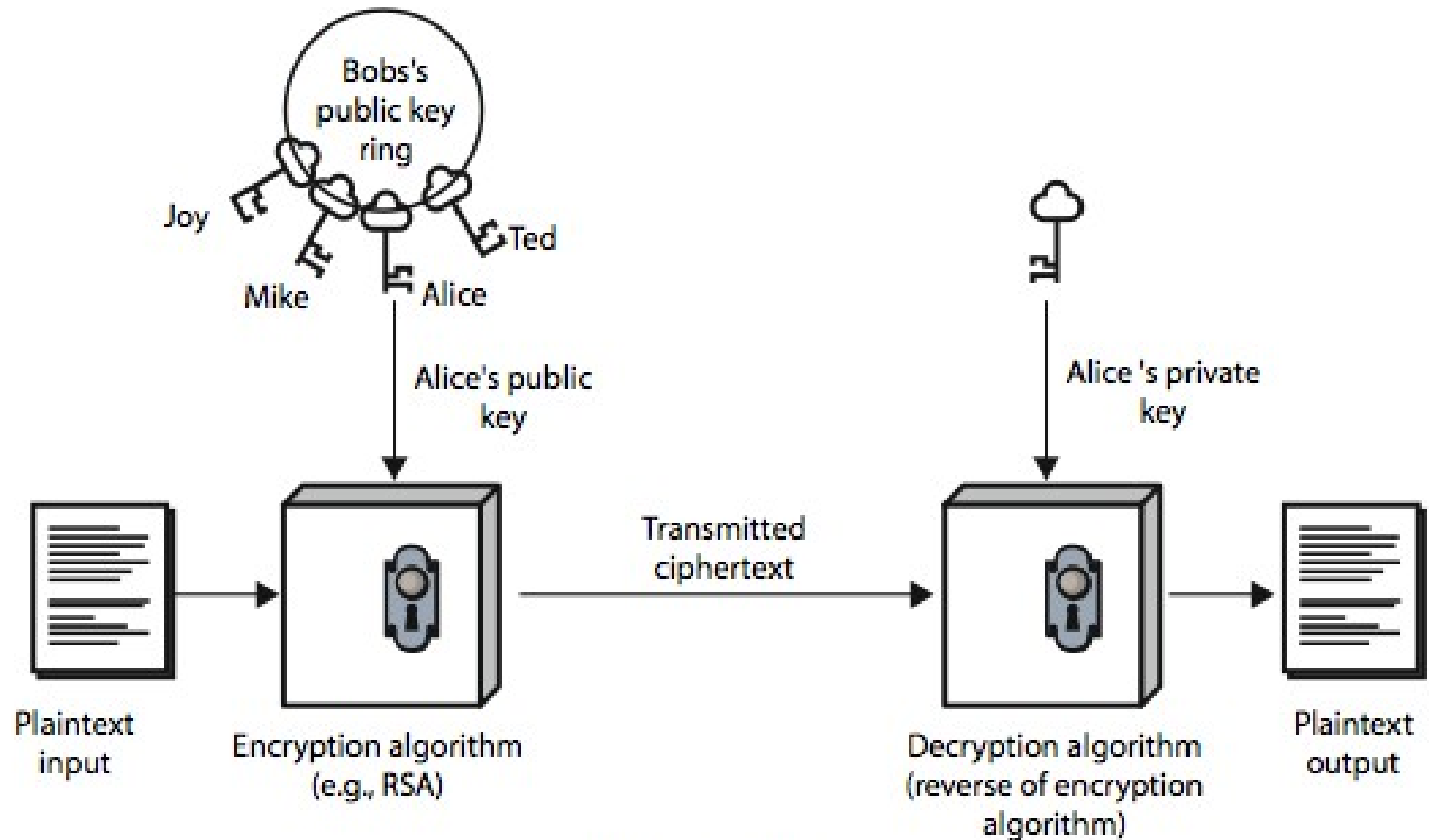
Why Public-Key Cryptography?

- developed to address two key issues:
 - **key distribution** – how to have secure communications in general without having to trust a KDC with your key
 - **digital signatures** – how to verify a message comes intact from the claimed sender.

Public-Key Cryptography

- **public-key/two-key/asymmetric** cryptography involves the use of **two** keys:
 - a **public-key**, which may be known by anybody, and can be used to **encrypt messages**, and **verify signatures**
 - a **private-key**, known only to the recipient, used to **decrypt messages**, and **sign (create) signatures**.

Public-Key Cryptography



(a) Encryption

RSA

- by Rivest, Shamir & Adleman of MIT in 1977
- best known & widely used public-key scheme
- based on exponentiation in a finite (Galois) field over integers modulo a prime
- uses large integers (e.g., 1024 bits)
- security due to cost of factoring large numbers

RSA Key Setup

- each user generates a public/private key pair by:
- selecting two large primes at random - p, q
- computing their system modulus $n = p \cdot q$
 - define $\phi(n) = (p-1)(q-1)$
- selecting at random the encryption key e
 - where $1 < e < \phi(n)$, $\gcd(e, \phi(n)) = 1$
- solve following equation to find decryption key d
 - $e \cdot d = 1 \pmod{\phi(n)}$ and $0 \leq d \leq n$
- publish their public encryption key: $KU = \{e, n\}$
- keep secret private decryption key: $KR = \{d, n\}$

RSA Use

- to encrypt a message M the sender:
 - obtains **public key** of recipient $KU = \{e, n\}$
 - computes: $C = M^e \bmod n$, where $0 \leq M < n$
- to decrypt the ciphertext C the owner:
 - uses their private key $KR = \{d, n\}$
 - computes: $M = C^d \bmod n$
- note that the message M must be smaller than the modulus n (block if needed)

RSA Example - Key Setup

1. Select primes: $p=17$ & $q=11$
2. Compute $n = pq = 17 \times 11 = 187$
3. Compute $\phi(n) = (p-1)(q-1) = 16 \times 10 = 160$
4. Select e : $\gcd(e, 160) = 1$; **choose** $e=7$
5. Determine d : $de=1 \pmod{160}$ **and** $d < 160$
Value is $d=23$ **since** $23 \times 7 = 161 = 10 \times 160 + 1$
6. Publish public key $KU = \{7, 187\}$
7. Keep secret private key $KR = \{23, 187\}$

Key Generation

Select p, q

p, q both prime, $p \neq q$

Calculate $n = p \times q$

Calculate $\phi(n) = (p-1) \times (q-1)$

Select integer e

$\gcd(\phi(n), e) = 1; 1 < e < \phi(n)$

Calculate d

Public key

$K_U = \{e, n\}$

Private key

$K_R = \{d, n\}$

Encryption

Plaintext:

$M < n$

Ciphertext:

$C = M^e \pmod{n}$

Decryption

Ciphertext:

C

Plaintext:

$M = C^d \pmod{n}$

RSA Example - En/Decryption

- sample RSA encryption/decryption is:

- given message $M = 88$

- encryption:

$$C = 88^7 \bmod 187 = 11$$

- decryption:

$$M = 11^{23} \bmod 187 = 88$$

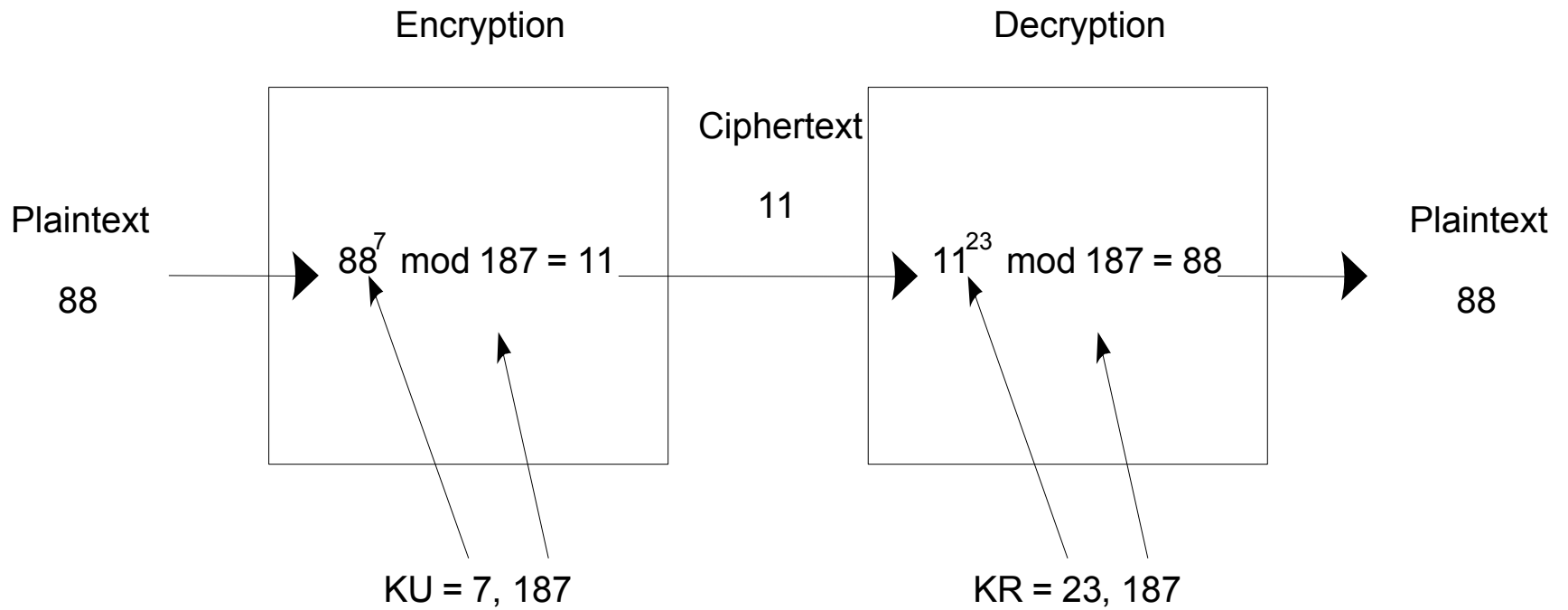


Figure 1. Example of RSA Algorithm

RSA Key Generation

- users of RSA must:
 - determine two primes at random - p, q
 - select either e or d and compute the other
- primes p, q must not be easily derived from modulus $n=p \cdot q$
 - means must be sufficiently large
 - typically guess and use probabilistic test
- exponents e, d are inverses, so use Inverse algorithm to compute the other